

VIT LOGO

COURSE NAME

BCSE320L – Web Application Security

CASE STUDY PROJECT REPORT

< Full Project / Case Study Title >

Group Number: _____

Group Members (Name – Reg. No.):

1. _____
2. _____
3. _____
4. _____
5. _____

Submitted to: < Faculty Name >

School of Computer Science and Engineering

< Institution Name >

< Month, Year >

DECLARATION AND ETHICAL-USE STATEMENT

We declare that this report is our own original work carried out as part of the BCSE320L Web Application Security course. All sources are cited. All security testing described in this report was performed only on applications we built ourselves or on sanctioned vulnerable-lab environments. No third-party, production, or unauthorised system was tested, and no real user data was accessed at any time.

TABLE OF CONTENTS

(Right-click and choose "Update Field" in Word to refresh page numbers after writing the report.)

DECLARATION AND ETHICAL-USE STATEMENT	2
TABLE OF CONTENTS	3
ABSTRACT	5
1. INTRODUCTION	6
1.1 Background	6
1.2 Motivation	6
1.3 Objectives of the Project	6
1.4 Scope of the Report	6
2. LITERATURE AND BACKGROUND STUDY	6
2.1 Related Concepts and Standards	6
2.2 Related Work / Existing Solutions	6
2.3 Gap Identified.....	7
3. PROBLEM STATEMENT AND SCOPE	7
3.1 Problem Definition	7
3.2 In-Scope and Out-of-Scope	7
3.3 Assumptions and Constraints	7
4. SYSTEM UNDER TEST AND ARCHITECTURE	7
4.1 Application Overview	7
4.2 Architecture Diagram	8
4.3 Frontend Description	8
4.4 Backend Description	8
4.5 Data Stores	8
4.6 Lab Setup and Tools.....	8
5. THREAT MODEL AND SECURITY REQUIREMENTS	8
5.1 Assets to Protect	8
5.2 Threat Actors	9
5.3 Threat Enumeration	9
5.4 Security Requirements.....	9

5.5 Success Criteria	9
6. METHODOLOGY	9
6.1 Reconnaissance.....	9
6.2 Vulnerability Identification	9
6.3 Exploitation	10
6.4 Remediation.....	10
6.5 Verification.....	10
7. IMPLEMENTATION AND EXPERIMENTS	10
7.1 Vulnerable Build	10
7.2 Findings	10
7.3 Defended Build.....	10
7.4 Test Cases.....	11
8. RESULTS AND ANALYSIS	11
8.1 Findings Summary.....	11
8.2 Before vs After Comparison.....	11
8.3 Performance Impact of Defences	11
8.4 Root-Cause Analysis	11
8.5 Discussion	12
9. LIMITATIONS	12
10. CONTEMPORARY ISSUES.....	12
11. CONCLUSION AND FUTURE WORK.....	12
11.1 Conclusion.....	12
11.2 Future Work.....	12
12. INDIVIDUAL CONTRIBUTIONS	13
13. REFERENCES	13
14. APPENDICES.....	13
REPORT FORMATTING CHECKLIST.....	14

ABSTRACT

What to include: 120–200 words. In one paragraph state: the target application and its context, the vulnerability class studied, the testing methodology and main tools, the key findings (how many issues and their severity), the countermeasures implemented, and the measured result after remediation. End with one major limitation. No citations, figures or abbreviations here.

Keywords:

< 4–6 keywords, e.g. web application security, XSS, CVSS, secure coding, OWASP >

1. INTRODUCTION

1.1 Background

What to include: Brief background on the web application security area your project sits in (reconnaissance, API security, injection, XSS/CSRF/XXE, DoS, credential handling, dependency security, or vulnerability management). Introduce the core concepts a reader needs.

1.2 Motivation

What to include: Why this problem matters: real-world impact, business and user risk, and one or two examples of public incidents or breaches in this category.

1.3 Objectives of the Project

What to include: Bullet list of what you set out to achieve — the security goals of the study and the specific course concepts applied. These must be measurable and are revisited in the Conclusion.

- < objective 1 >
- < objective 2 >
- < objective 3 >

1.4 Scope of the Report

What to include: One sentence per remaining section describing how the report is organised.

2. LITERATURE AND BACKGROUND STUDY

2.1 Related Concepts and Standards

What to include: Explain the standards and references you rely on: OWASP Top 10, OWASP Testing Guide (WSTG), CWE, CVSS, and the relevant chapter of the Hoffman textbook.

2.2 Related Work / Existing Solutions

What to include: Summarise 3–5 prior works, tools or known techniques relevant to your vulnerability class, in your own words with citations. A short comparison table is encouraged.

Ref.	Approach / Tool	Strengths	Limitations
[1]			
[2]			

Ref.	Approach / Tool	Strengths	Limitations
[3]			

2.3 Gap Identified

What to include: One short paragraph: what is missing or worth demonstrating that your project addresses.

3. PROBLEM STATEMENT AND SCOPE

Restate the chosen problem/scenario in your own words. Then define scope precisely.

3.1 Problem Definition

What to include: A clear statement of the security problem your full-stack application demonstrates and defends against.

3.2 In-Scope and Out-of-Scope

What to include: Two bullet lists: features and attack classes that ARE covered, and those explicitly NOT covered.

In scope:

- < ... >

Out of scope:

- < ... >

3.3 Assumptions and Constraints

- Attacker model: unauthenticated outsider / authenticated low-privilege user / insider / supply-chain.
- Access level: black-box / grey-box / white-box (is source code available?).
- Environment: local lab / container / cloud; network reachability.
- Constraints: time, tool licences, no destructive or production testing.

4. SYSTEM UNDER TEST AND ARCHITECTURE

Describe the full-stack application you built — the thing being attacked and defended.

4.1 Application Overview

What to include: What the app does, its main features, and the user roles. Include a labelled screenshot of the working UI.

4.2 Architecture Diagram

What to include: A clear diagram showing frontend, backend, database, and any external services, with communication paths and trust boundaries marked. Caption it as Figure 1.

4.3 Frontend Description

What to include: Framework/library and version, pages/components, how state and sessions are handled, and every client-side data store used (localStorage, sessionStorage, cookies, IndexedDB).

4.4 Backend Description

What to include: Language/runtime and framework with versions, the web/application server, API style (REST/GraphQL/RPC), and the endpoint inventory table below.

Method	Path	Parameters	Auth?	Data sensitivity
GET	/api/...			
POST	/api/...			

4.5 Data Stores

What to include: Database type and schema summary; what sensitive data is stored and where; how secrets/credentials are handled.

4.6 Lab Setup and Tools

What to include: VM/container specs, IP addresses and ports, and the exact tools with versions (Burp Suite, OWASP ZAP, sqlmap, Nikto, ffuf/gobuster, Nuclei, OWASP Dependency-Check/Snyk, SonarQube/Semgrep, Wireshark, browser DevTools). State how a reader reproduces the environment — put setup scripts/docker-compose in Appendix D.

5. THREAT MODEL AND SECURITY REQUIREMENTS

5.1 Assets to Protect

What to include: List assets: credentials, session tokens, PII, business data, availability.

5.2 Threat Actors

What to include: Who might attack and what capabilities they have.

5.3 Threat Enumeration

What to include: Enumerate threats using STRIDE or the OWASP Top 10 as a checklist, presented as the table below.

ID	Threat / STRIDE category	Affected component	Likelihood	Impact
T1				
T2				
T3				

5.4 Security Requirements

- Functional: authentication, authorization, input validation, output encoding, logging, rate limiting.
- Non-functional: confidentiality, integrity, availability targets; acceptable performance overhead for defences.

5.5 Success Criteria

What to include: How you will judge success, e.g. "all High/Critical findings non-exploitable after remediation" or "scanner alerts reduced from N to M".

6. METHODOLOGY

What to include: State the methodology followed — OWASP WSTG, PTES, or the textbook's reconnaissance → attack → defence cycle — then describe each phase. Include a methodology flow diagram (Figure 2).

6.1 Reconnaissance

What to include: Information gathering, application mapping, endpoint/subdomain discovery, framework fingerprinting.

6.2 Vulnerability Identification

What to include: Manual testing plus automated scanning; how you triaged results and removed false positives.

6.3 Exploitation

What to include: How you built controlled proof-of-concept exploits for each confirmed finding.

6.4 Remediation

What to include: The code and configuration changes made in the defended build.

6.5 Verification

What to include: How you re-tested to confirm each fix and checked for regressions.

7. IMPLEMENTATION AND EXPERIMENTS

7.1 Vulnerable Build

What to include: Describe how the vulnerable version was implemented and which weaknesses it contains. Reference the exact code locations.

7.2 Findings

Document EACH finding in this fixed format so findings are comparable:

- Finding ID and title
- Affected component / endpoint
- Vulnerability class and CWE identifier
- CVSS v3.1 base vector, score and severity — show the vector string and justify each metric
- Preconditions to exploit
- Proof of concept — exact request/payload plus a screenshot of the result
- Business impact in plain language
- Remediation applied — before/after code snippet
- Verification evidence that the fix works

7.3 Defended Build

Describe the countermeasures implemented, grouped by layer:

- Code-level: parameterised queries, output encoding, allow-list validation, safe parser/regex.
- Configuration-level: security headers (CSP, HSTS, X-Content-Type-Options, Referrer-Policy), cookie flags (Secure, HttpOnly, SameSite), TLS configuration.

- Architectural: anti-CSRF tokens, rate limiting, least-privilege DB accounts, dependency pinning.
- Automation: SAST/DAST/SCA in the pipeline; regression tests for each fixed vulnerability.

7.4 Test Cases

What to include: Table of test cases with result before and after the fix.

TC ID	Objective	Input / Payload	Before fix	After fix	Pass?
TC1					
TC2					
TC3					

8. RESULTS AND ANALYSIS

8.1 Findings Summary

What to include: Summary table of all findings with a severity distribution (Critical/High/Medium/Low) and a chart (Figure 3).

Finding	Class / CWE	CVSS	Severity	Status
F1				
F2				
F3				

8.2 Before vs After Comparison

What to include: Exploitable findings, scanner alert counts and total CVSS before and after remediation — with a comparison chart.

8.3 Performance Impact of Defences

What to include: Latency or throughput measured before and after adding defences; state number of runs and averaging method.

8.4 Root-Cause Analysis

What to include: For each vulnerability, why it existed — the root cause, linked to secure-coding anti-patterns, not just the symptom.

8.5 Discussion

What to include: Any defence that was bypassed or only partially effective, and why. Honest analysis earns marks.

9. LIMITATIONS

- Coverage: what was not tested and why.
- Tools: false negatives, community-edition restrictions.
- Environment: lab scale vs production reality; no load-realistic DoS testing.
- Validity: single environment, limited payload sample.

10. CONTEMPORARY ISSUES

What to include: One or two recent real-world incidents, CVEs or advisories connected to your topic (ideally 2024–2026). Explain what happened and which defence from your project would have prevented it. Mention emerging concerns where relevant — supply-chain attacks, prompt injection in LLM-backed apps, API-first attack surface, dependency confusion.

11. CONCLUSION AND FUTURE WORK

11.1 Conclusion

What to include: What was demonstrated, stated directly against each objective from Section 1.3. Summarise the key security lesson — about secure design, not just the specific bug.

11.2 Future Work

What to include: Concrete next steps: additional vulnerability classes, wider automation, production hardening.

12. INDIVIDUAL CONTRIBUTIONS

What to include: Table of each member's sections owned, tasks performed and approximate percentage. Signed by all members.

Name	Reg. No.	Sections / tasks owned	% contribution

13. REFERENCES

What to include: IEEE format, numbered in order of citation. Minimum 10 references, at least 5 from peer-reviewed or official sources. Must include the textbook and any OWASP/CWE/NVD identifiers cited.

[1] A. Hoffman, Web Application Security: Exploitation and Countermeasures for Modern Web Applications, 1st ed. Sebastopol, CA: O'Reilly Media, 2020.

[2] OWASP Foundation, "OWASP Top 10," [Online]. Available: ... (cite the specific version/year).

[3] < add remaining references >

14. APPENDICES

- Appendix A — Full source code of vulnerable and patched builds (or repository link).
- Appendix B — Raw tool output and scan reports.
- Appendix C — Complete payload list.
- Appendix D — Environment setup scripts (docker-compose / commands).
- Appendix E — Signed ethical-use declaration and list of systems tested.

REPORT FORMATTING CHECKLIST

- Body font: Times New Roman 12 pt or Calibri 11 pt; 1.5 line spacing; justified.
- Headings numbered exactly as in this template; do not rename or reorder sections.
- Every figure and table numbered and captioned; every figure referenced in the text.
- Every claim, payload or statistic taken from another source must be cited.
- Report submitted as PDF, plus the source repository, demo and Power point slides.
- Update the Table of Contents and figure/table lists before exporting to PDF.